SYSTEMDOKUMENTATION & ARCHITEKTUR

# Parky – Intermodales P+R Routing

**Version:** 1.0.0
**Stack:** .NET 8 / JS / Leaflet
**Hosting:** Render & Vercel

## 1. Projektübersicht & Zielsetzung

**Parky** ist eine intermodale Navigationsanwendung, die Autofahrt und öffentlichen Personennahverkehr (ÖPNV) optimal kombiniert. Das System berechnet ausgehend von einem Startpunkt den idealen Park-and-Ride (P+R) Parkplatz und verknüpft die Autofahrt nahtlos mit den besten ÖPNV-Verbindungen zum Zielort.

## 2. Systemarchitektur & Infrastruktur

Die Anwendung ist in eine entkoppelte Client-Server-Architektur unterteilt, um eine maximale Skalierbarkeit und einfache Bereitstellung zu gewährleisten:

| Komponente | Technologie | Hosting / Provider | Beschreibung |
|---|---|---|---|
| **Frontend** | HTML5, CSS3, JavaScript (ES6+), Leaflet.js | Vercel | Single-Page-Application (SPA) mit interaktiver Kartenanzeige, dynamischen Routenkarten und LocalStorage-Historie. |
| **Backend API** | C# .NET 8 Web API (`RouteController`, `ParkAndRideController`) | Render (Web Service) | Verarbeitet Routing-Anfragen, berechnet Intermodal-Scores und aggregiert Parkplatz- sowie ÖPNV-Daten. |
| **Geocoding** | OpenStreetMap Nominatim API | External API | Wandelt Freitext-Adressen im Frontend in Geokoordinaten (Breiten- und Längengrad) um. |
| **Routing-Dienste** | OSRM (Driving & Foot) / VBB Transit API | External APIs | Liefert Straßen-Polylines für den Pkw-Teil sowie Fahrplandaten für den ÖPNV-Teil der Route. |

## 3. Schnittstellen & API-Spezifikation

Das Backend stellt zwei Haupt-Endpunkte zur Verfügung:

- `GET /api/parkandride`: Nimmt Start- und Zielkoordinaten auf und gibt eine nach Relevanz sortierte Liste passender P+R-Parkplätze zurück.
- `POST /api/Route/calculate`: Erhält Start-, Ziel- und Zeitparameter, ermittelt den besten P+R-Knotenpunkt und berechnet die vollständige Wege-Kette (Auto + Parken + ÖPNV + Fußweg).

> **Wichtige technische Erkenntnis zur Koordinaten-Formatierung**
> Parameter für Breitengrad ( `lat` ) und Längengrad ( `lon` ) müssen zwingend mit Kultur-Invarianz ( `CultureInfo.InvariantCulture` ) formatiert werden. Dies verhindert, dass Dezimaltrennpunkte in Ländern wie Deutschland fälschlicherweise als Kommas formatiert werden, was zu HTTP 400 Validation Errors führt.

## 4. Frontend-Funktionen & State Management

- **Interaktive Routendarstellung:** Farbcodierte Segmentierung auf Leaflet-Karten (Hellblau für Auto, Lila für ÖPNV, Orange gestrichelt für Fußwege).
- **Suchhistorie ("Neulich gesucht"):** Speicherung der letzten 5 Suchanfragen im `localStorage` des Browsers. Wiederherstellung der Suchparameter und Ausführung der Route per Klick.
- **Toast-Benachrichtigungen:** Asynchrone Rückmeldung des Berechnungs- und API-Status direkt an den Benutzer.

## 5. Fehlerbehebung & Betriebs-Guidelines

### Cold-Start Behandlung (Render Free Tier)

Da Render-Instanzen im kostenlosen Tarif nach Inaktivität in den Schlafmodus wechseln, kann der erste Aufruf von `/api/Route/calculate` in einen Timeout (HTTP 500) laufen. Ein automatisches Keep-Alive oder manueller Warm-Up Ping vor Präsentationen verhindert Ausfälle.

### Git Version Control & Merge-Konflikte

Um Synchronisationsprobleme bei der Teamarbeit zu verhindern, sind Build-Artefakte (`bin/`, `obj/`, `.vs/`) strikt in der `.gitignore` eingetragen. Bei Konflikten in generierten DLLs empfiehlt sich ein Zurücksetzen mittels `git merge --abort` und anschließendem Entfernen der temporären Ordner.